

Window Functions in SQL

- Beginner Friendly Introduction
- Sphoorthi Free Classes - Data Engineering

What are Window Functions?

- Window functions perform calculations across a set of rows related to the current row.
- Unlike GROUP BY:
 - - GROUP BY collapses rows
 - - Window functions keep original rows

Why are Window Functions Important?

- Used heavily in:
 - - Data Engineering
 - - Analytics
 - - Dashboards
 - - ETL Pipelines
 - - Business KPIs

- Common uses:

Sphoorthi Free Classes - Data Engineering

- - Ranking

Basic Syntax

- SELECT column_name,
- WINDOW_FUNCTION() OVER (
- PARTITION BY column_name
- ORDER BY column_name
-)
- FROM table_name;

Sample Sales Table

- Columns:
- - order_id
- - customer_name
- - city
- - product_name
- - amount
- - order_date

ROW_NUMBER()

- Assigns unique sequential numbers.
- Example:
- `SELECT customer_name,`
- `amount,`
- `ROW_NUMBER() OVER(ORDER BY amount`
`DESC) AS rn`
- `FROM sales;`

RANK()

- Assigns ranks with gaps.
- If two rows tie:
 - - both get same rank
 - - next rank is skipped

DENSE_RANK()

- Assigns ranks without gaps.
- Example:
- 1, 2, 2, 3
- instead of
- 1, 2, 2, 4

PARTITION BY

- Creates groups inside window functions.
- Example:
- Find highest sales inside each city.
- SELECT city,
- customer_name,
- RANK() OVER(PARTITION BY city ORDER BY
amount DESC)

ORDER BY inside OVER()

- Controls ordering inside the window.
- Example:
- `ROW_NUMBER() OVER(ORDER BY amount DESC)`
- Highest amount gets row number 1

Running Total

- Example:
- SELECT order_date,
- amount,
- SUM(amount) OVER(ORDER BY
order_date)
- AS running_total
- FROM sales;

Deduplication Example

- Very important in ETL.
- SELECT *,
- ROW_NUMBER() OVER(
- PARTITION BY customer_name
- ORDER BY order_date DESC
-) AS rn
- FROM sales;

Difference: GROUP BY vs Window Functions

- GROUP BY:
 - - Combines rows
 - - Reduces row count
- Window Functions:
 - - Preserve rows
 - - Add analytical calculations

Common Window Functions

- - ROW_NUMBER()
- - RANK()
- - DENSE_RANK()
- - SUM()
- - AVG()
- - MIN()
- - MAX()
- - LEAD()

- - LAG()

LEAD() and LAG()

- Used to compare rows.
- LEAD() -> next row
- LAG() -> previous row
- Helpful for trend analysis.

Real World Uses

- - Top customers
- - Highest selling products
- - Latest transactions
- - Duplicate removal
- - Monthly revenue tracking
- - Employee ranking

Key Points to Remember

- - OVER() is mandatory
- - PARTITION BY creates groups
- - ORDER BY controls sequence
- - Rows are NOT collapsed
- - Very common in ETL & Analytics